

ΤΕΧΝΙΚΗ ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΑΝΑΛΥΣΗ ΛΟΓΙΣΜΙΚΟΥ

Έργο: AEGIS-VIO

Uncertainty-Aware Visual-Inertial Navigation in Challenging Environments

Αποθετήριο: github.com/PanagiotaGr/aegis-vio

Συγγραφέας αποθετηρίου: Panagiota Grosdoulis

Έγγραφο τεχνικής/ακαδημαϊκής ανάλυσης λογισμικού

Εκπονήθηκε ως πλήρης μελέτη αρχιτεκτονικής, σχεδιαστικών αποφάσεων και ποιότητας κώδικα

Πίνακας Περιεχομένων

- 1. Εκτελεστική Περίληψη (Executive Summary)**
- 2. Κίνητρο του Έργου (Project Motivation)**
- 3. Συνολική Αρχιτεκτονική (Overall Architecture)**
- 4. Ανάλυση Καταλόγων (Folder-by-Folder Analysis)**
 - 4.1 src/
 - 4.2 ros2_ws/src/aegis_vio/
 - 4.3 scripts/
 - 4.4 tests/
 - 4.5 docs/
 - 4.6 datasets/, models/, plots/, results/
 - 4.7 web/
 - 4.8 .github/workflows/

1. Εκτελεστική Περίληψη (Executive Summary)

Το αποθετήριο AEGIS-VIO (“Uncertainty-Aware Visual-Inertial Navigation in Challenging Environments”) αποτελεί ένα ερευνητικό πλαίσιο (research framework) γραμμένο κατά κύριο λόγο σε Python, το οποίο διερευνά μια συγκεκριμένη ερευνητική ιδέα στον χώρο της ρομποτικής αντίληψης: ότι ένα σύστημα Visual-Inertial Odometry (VIO) δεν πρέπει απλώς να παράγει μια εκτίμηση θέσης ανά καρέ, αλλά πρέπει επίσης να γνωρίζει πόσο πρέπει να εμπιστεύεται την ίδια του την εκτίμηση, και να μεταφράζει αυτή τη γνώση σε συγκεκριμένη συμπεριφορά πλοήγησης (μείωση ταχύτητας, διεύρυνση ζώνης ασφαλείας, διακοπή κίνησης).

Το έργο δεν είναι ένα πλήρως ολοκληρωμένο, παραγωγικής ποιότητας (production-grade) σύστημα SLAM/VIO όπως τα ORB-SLAM3, VINS-Mono ή OpenVINS. Είναι, όπως δηλώνεται ρητά και στο README του αποθετηρίου, ένα “research prototype”: ένα πειραματικό υπόβαθρο πάνω στο οποίο δοκιμάζεται η θεμελιακή ιδέα της “uncertainty-aware autonomy”, με σαφή τεκμηρίωση των ορίων της τρέχουσας υλοποίησης και ρητό οδικό χάρτη (roadmap) για μελλοντική επέκταση προς πλήρη back-end πολλαπλών όψεων (multi-view) με τριγωνισμό σημείων-αναφοράς (landmarks).

Το πρόβλημα που επιχειρεί να θεραπεύσει το AEGIS-VIO είναι ένα πραγματικό και τεκμηριωμένο πρόβλημα στη βιβλιογραφία της αυτόνομης πλοήγησης: τα συστήματα VIO/SLAM συνεχίζουν να εκπέμπουν εκτιμήσεις θέσης ακόμη και όταν οι συνθήκες αντίληψης (κίνηση θόλωσης, χαμηλός φωτισμός, διαδρομές με λίγα οπτικά χαρακτηριστικά, απότομες κινήσεις) έχουν υποβαθμίσει σημαντικά την ποιότητα της εκτίμησης. Η απουσία ενός μηχανισμού “παρακολούθησης της αξιοπιστίας” της ίδιας της εκτίμησης μπορεί να οδηγήσει έναν αυτόνομο ρομποτικό πράκτορα να συνεχίσει να κινείται με πλήρη ταχύτητα ενώ στην πραγματικότητα “δεν ξέρει πού βρίσκεται”, με ενδεχόμενο αποτέλεσμα σύγκρουση ή απώλεια εντοπισμού.

Οι βασικοί χρήστες-στόχοι του πλαισίου είναι ερευνητές και μεταπτυχιακοί/διδακτορικοί φοιτητές ρομποτικής και υπολογιστικής όρασης που μελετούν active perception και risk-aware autonomy, καθώς και μηχανικοί που θέλουν ένα ελαφρύ, κατανοητό υπόβαθρο πάνω στο οποίο θα χτίσουν πιο σύνθετους αλγορίθμους αβεβαιότητας (π.χ. learned uncertainty prediction με νευρωνικά δίκτυα).

Τα κύρια χαρακτηριστικά του συστήματος, όπως προκύπτουν από την ανάλυση του πηγαίου κώδικα, είναι: (α) φόρτωση και ανάγνωση δεδομένων του πρότυπου dataset EuRoC MAV, (β) ένα ελάχιστο αλλά λειτουργικό Extended Kalman Filter (EKF) για εκτίμηση κατάστασης, (γ) ένας ανιχνευτής/αντιστοιχιστής οπτικών χαρακτηριστικών βασισμένος σε ORB με γεωμετρικό φιλτράρισμα RANSAC, (δ) μια ολοκληρωμένη σουίτα μετρικών αβεβαιότητας (ίχνος και ορίζουσα πίνακα συνδιακύμανσης, εντροπία, απόσταση Mahalanobis), (ε) έναν ελεγκτή πλοήγησης που μετατρέπει την

αβεβαιότητα σε διακριτές καταστάσεις συμπεριφοράς, (στ) εργαλεία αξιολόγησης ακρίβειας τροχιάς (ATE), (ζ) ένα πλήρες, αν και σε πρώιμο στάδιο, πακέτο ROS2 για λειτουργία σε πραγματικό ρομπότ, και (η) ένα διαδραστικό, καθαρά εκπαιδευτικό demo σε JavaScript/HTML που οπτικοποιεί τη λογική “αβεβαιότητα → συμπεριφορά” μέσα στον φυλλομετρητή.

Η γενική φιλοσοφία του λογισμικού είναι η διάκριση ανάμεσα σε ένα πυρήνα ανεξάρτητο από το ROS2 (στον κατάλογο src/), που μπορεί να ελεγχθεί μονάδα προς μονάδα με pytest χωρίς να απαιτείται εγκατάσταση ROS2, και ένα περιβάλλον ολοκλήρωσης (ros2_ws/) που προορίζεται για πραγματική ή προσομοιωμένη λειτουργία ρομπότ. Αυτή η αρχιτεκτονική απόφαση -ο διαχωρισμός “αλγορίθμου” από “μέσο εκτέλεσης”- είναι ευρέως αποδεκτή καλή πρακτική στη ρομποτική και αποτελεί ένα από τα πιο θετικά στοιχεία του σχεδιασμού.

Από τεχνολογική άποψη, ο πυρήνας βασίζεται σε NumPy/SciPy για γραμμική άλγεβρα, OpenCV για όραση υπολογιστών (ORB feature detection, RANSAC), pandas για διαχείριση δεδομένων σε μορφή πίνακα, Matplotlib για οπτικοποίηση, scikit-learn για ένα πειραματικό μοντέλο πρόβλεψης αποτυχίας εντοπισμού βασισμένο σε Random Forest, pytest για δοκιμές μονάδας, και ROS2 (rclpy) για την επικάλυψη πάνω στο μεσισμικό λογισμικό ρομποτικής. Στο επίπεδο της διεπαφής χρήστη υπάρχει ένα απλό, αυτόνομο HTML/CSS/JavaScript demo που δεν εξαρτάται από τον πυρήνα Python.

Ως κύριο πλεονέκτημα του έργου αναδεικνύεται η εννοιολογική του καθαρότητα και η ειλικρινής τεκμηρίωση της τρέχουσας κατάστασής του: το README δηλώνει ρητά ότι το measurement update στο vio_estimator.py είναι “intentionally simplified, clearly documented placeholder”, κάτι σπάνιο σε ερευνητικά αποθετήρια, όπου συχνά η τεκμηρίωση υπερεκτιμά την πραγματική ωριμότητα του κώδικα. Αυτή η ειλικρίνεια διευκολύνει σημαντικά την αξιολόγηση και την περαιτέρω ανάπτυξη του έργου από τρίτους.

2. Κίνητρο του Έργου (Project Motivation)

Η ανάπτυξη συστημάτων VIO/SLAM αποτελεί θεμελιακό πρόβλημα της αυτόνομης ρομποτικής, με εφαρμογές από μη επανδρωμένα εναέρια οχήματα (drones) μέχρι αυτόνομα οχήματα εδάφους και φορητές συσκευές augmented reality. Η πλειονότητα των δημόσια διαθέσιμων λύσεων (π.χ. ORB-SLAM3, VINS-Fusion, OpenVINS) επικεντρώνεται στη μεγιστοποίηση της ακρίβειας της εκτιμώμενης τροχιάς, αναφέροντας μετρικές όπως το Absolute Trajectory Error (ATE). Ωστόσο, η ακρίβεια από μόνη της δεν αρκεί σε σενάρια πραγματικής αυτονομίας: ένα ρομπότ που δεν γνωρίζει πότε η εκτίμησή του είναι αναξιόπιστη δεν μπορεί να αποφασίσει σωστά πότε πρέπει να επιβραδύνει, να σταματήσει ή να αναζητήσει καλύτερη οπτική γωνία.

Το AEGIS-VIO τοποθετείται ακριβώς σε αυτό το κενό. Αντί να ανταγωνίζεται τα υπάρχοντα συστήματα ως προς την ακρίβεια εντοπισμού, εστιάζει στο πώς η αβεβαιότητα μιας εκτίμησης (την οποία ο πίνακας συνδιακύμανσης ενός φίλτρου Kalman ήδη περιέχει εγγενώς) μπορεί να μετασχηματιστεί σε ενιαία, ερμηνεύσιμη βαθμολογία κινδύνου (risk score) και να τροφοδοτήσει άμεσα τη λήψη αποφάσεων πλοήγησης. Πρόκειται για μια θεωρητικά θεμελιωμένη προσέγγιση: η συνδιακύμανση ενός EKF αποτελεί ήδη ποσοτική αναπαράσταση της αβεβαιότητας, αλλά σπάνια χρησιμοποιείται ενεργά στη ροή ελέγχου του ρομπότ -συνήθως καταναλώνεται μόνο παθητικά, για παράδειγμα στην οπτικοποίηση ελλειψοειδών εμπιστοσύνης.

Εναλλακτικές προσεγγίσεις στο ίδιο πρόβλημα περιλαμβάνουν: (α) heuristics βασισμένες σε κανόνες πάνω στον αριθμό ανιχνευμένων χαρακτηριστικών (π.χ. “αν matches < threshold, σταμάτα”), οι οποίες είναι απλές αλλά αγνοούν τη δυναμική εξέλιξη της αβεβαιότητας μέσα στον χρόνο, (β) πλήρως μαθησιακές (learning-based) προσεγγίσεις πρόβλεψης αποτυχίας απευθείας από εικόνα, οι οποίες απαιτούν μεγάλο όγκο επισημειωμένων δεδομένων αποτυχίας και είναι δυσκολότερο να ερμηνευθούν, και (γ) πλήρη πιθανοτικά πλαίσια Factor Graph (π.χ. GTSAM) με marginalization βασισμένα σε αβεβαιότητα, τα οποία είναι θεωρητικά ισχυρότερα αλλά πολύ πιο σύνθετα στην υλοποίηση και τη συντήρηση.

Η επιλογή του AEGIS-VIO να ξεκινήσει από ένα απλό, διαφανές EKF και μια διαφανή, παραμετροποιήσιμη πολιτική ορίων (threshold policy) έχει το πλεονέκτημα της ερμηνευσιμότητας: κάθε μετάβαση κατάστασης πλοήγησης (NORMAL → CAUTIOUS → RECOVERY → HALT) μπορεί να εξηγηθεί με σαφήνεια μέσω συγκεκριμένων αριθμητικών ορίων, πράγμα κρίσιμο σε ένα ερευνητικό πλαίσιο όπου ο στόχος είναι να αποδειχθεί η ίδια η ιδέα, και όχι να μεγιστοποιηθεί η απόδοση με ένα “μαύρο κουτί”.

Επιπρόσθετα, η ύπαρξη ενός παράλληλου, πειραματικού μονοπατιού μηχανικής μάθησης (το train_failure_predictor.py που εκπαιδεύει έναν Random Forest classifier πάνω σε χαρακτηριστικά αβεβαιότητας για να προβλέψει μελλοντική αποτυχία

εντοπισμού) δείχνει ότι οι συγγραφείς αντιμετωπίζουν τη threshold-based προσέγγιση ως ένα πρώτο, ερμηνεύσιμο βήμα, με σκόπιμη πρόθεση να συγκριθεί αργότερα με πιο σύνθετες, μαθησιακές μεθόδους πρόβλεψης.

3. Συνολική Αρχιτεκτονική (Overall Architecture)

Το σύστημα οργανώνεται σε τέσσερα διακριτά επίπεδα, καθένα με σαφή ευθύνη και σαφή διεπαφή προς τα υπόλοιπα: (1) το επίπεδο πυρήνα αλγορίθμων (src/), ανεξάρτητο πλατφόρμας εκτέλεσης, (2) το επίπεδο σεναρίων εκτέλεσης πειραμάτων (scripts/), που ενορχηστρώνει τον πυρήνα πάνω σε πραγματικά ή συνθετικά δεδομένα, (3) το επίπεδο ολοκλήρωσης σε ρομποτικό μεσισμικό (ros2_ws/), που προσφέρει διεπαφή πραγματικού χρόνου, και (4) το επίπεδο εκπαιδευτικής/παρουσιαστικής διεπαφής (web/), μια αυτοτελής εφαρμογή φυλλομετρητή που αναπαριστά εννοιολογικά τη λογική του συστήματος χωρίς να καλεί τον πραγματικό κώδικα Python.

Δεν υπάρχει στην τρέχουσα έκδοση παραδοσιακό “backend” με REST API ή βάση δεδομένων, ούτε σύστημα ελέγχου πρόσβασης χρηστών. Πρόκειται για ένα επιστημονικό/ρομποτικό λογισμικό και όχι για διαδικτυακή εφαρμογή πολλών χρηστών, οπότε η απουσία αυτών των στοιχείων δεν αποτελεί έλλειψη αλλά ορθή αντιστοίχιση αρχιτεκτονικής στον σκοπό του έργου. Η “επικοινωνία” στο σύστημα νοείται ως ροή δεδομένων ανάμεσα σε αρθρωτές (modules) Python, και, στο επίπεδο ROS2, ως ροή μηνυμάτων ανάμεσα σε κόμβους (nodes) μέσω του πρωτοκόλλου δημοσίευσης/συνδρομής (publish/subscribe) του ROS2.

Η ροή δεδομένων στο επίπεδο πυρήνα έχει την ακόλουθη λογική αλληλουχία, όπως προκύπτει από τον συνδυασμό των αρχείων `feature_tracker.py`, `ekf.py` / `vio_estimator.py`, `uncertainty.py` και `uncertainty_aware_controller.py` / `navigation.py`: πρώτα δύο διαδοχικές εικόνες περνούν από τον ανιχνευτή χαρακτηριστικών ORB, ο οποίος παράγει έναν αριθμό αντιστοιχίσεων (matches) και έναν λόγο εσωτερικών σημείων (inlier ratio) μέσω RANSAC. Το γινόμενο αυτών των δύο μεγεθών ορίζεται ως δείκτης οπτικής ποιότητας (visual quality). Παράλληλα, το φίλτρο Kalman προβλέπει (predict) την επόμενη κατάσταση υπό υπόθεση σταθερής ταχύτητας και ενημερώνεται (update) όταν υπάρχει νέα μέτρηση θέσης, με τον θόρυβο μέτρησης να εξαρτάται αντιστρόφως από την ποιότητα του οπτικού καναλιού -μια άμεση και κρίσιμη σύζευξη ανάμεσα στο front-end όρασης και το back-end εκτίμησης κατάστασης.

Στη συνέχεια, ο πίνακας συνδιακύμανσης P του φίλτρου τροφοδοτεί το άρθρωμα `uncertainty.py`, το οποίο υπολογίζει το ίχνος (trace), τη διαφορική εντροπία και, όπου χρειάζεται, την απόσταση Mahalanobis. Ο δείκτης `uncertainty_score` (ίχνος διαιρεμένο με την ποιότητα) μετατρέπεται σε κανονικοποιημένη βαθμολογία κινδύνου $risk \in [0,1]$ στο επίπεδο των σεναρίων προσομοίωσης, η οποία στη συνέχεια περνά από μια κατωφλιωτή πολιτική απόφασης (ModePolicy ή UncertaintyAwareController) που παράγει μία από τις διακριτές καταστάσεις πλοήγησης και τα αντίστοιχα πολλαπλασιαστικά συντελεστές ταχύτητας και ζώνης ασφαλείας.

Στο επίπεδο ROS2, η ίδια λογική αλληλουχία αναπαρίσταται ως τρεις ξεχωριστοί κόμβοι: ο VIONode (εκτίμηση κατάστασης), ο UncertaintyNode (ποσοτικοποίηση αβεβαιότητας) και ο NavigationNode (απόφαση συμπεριφοράς), οι οποίοι θα επικοινωνούσαν μέσω προσαρμοσμένων μηνυμάτων `StateEstimate.msg` και `UncertaintyMetrics.msg`. Στην τρέχουσα έκδοση του αποθετηρίου, οι τρεις κόμβοι είναι σκελετοί (scaffolds): αρχικοποιούν έναν κόμβο ROS2 και γράφουν ένα μήνυμα καταγραφής, χωρίς να έχουν ακόμη συνδεθεί με τα modules του πυρήνα src/ μέσω πραγματικών publishers/subscribers. Αυτό είναι ξεκάθαρα δηλωμένο στο docs/usage.md (“The ROS2 nodes are initial scaffolds and will be connected to the Python research modules as the estimator matures”) και αποτελεί σημαντικό στοιχείο για την ορθή αποτίμηση της τρέχουσας ωριμότητας του έργου.

Δεν υπάρχει έννοια αυθεντικοποίησης (authentication) ή εξουσιοδότησης (authorization) χρηστών στο σύστημα, καθώς δεν πρόκειται για πολυχρηστική εφαρμογή· το μόνο στοιχείο που προσεγγίζει “ασφάλεια” είναι η ρομποτική έννοια της ασφαλούς λειτουργίας (safety), δηλαδή οι μηχανισμοί μείωσης ταχύτητας και διεύρυνσης ζώνης ασφαλείας υπό αβεβαιότητα, οι οποίοι αναλύονται διεξοδικά στην Ενότητα 10.

4. Ανάλυση Καταλόγων (Folder-by-Folder Analysis)

4.1 src/

Ο κατάλογος `src/` αποτελεί την “καρδιά” του έργου: έναν πλήρως ανεξάρτητο από το ROS2 πακέτο Python, εγκαθιστάμενο μέσω `setup.py` με `pip install -e .`, που περιέχει όλη την επιστημονική/αλγοριθμική λογική. Η ύπαρξή του ως ξεχωριστού, εγκαθιστάμενου πακέτου (και όχι απλώς ως φάκελου σεναρίων) επιτρέπει να εισάγεται με `from src.X import Y` από οπουδήποτε στο αποθετήριο -σενάρια, δοκιμές μονάδας, ή ενδεχομένως κόμβοι ROS2 στο μέλλον- χωρίς προβλήματα διαδρομών (path issues). Αυτή είναι μια καλή πρακτική οργάνωσης πακέτων Python.

Ο υποκατάλογος `src/simulation/` φιλοξενεί αποκλειστικά τη συνθετική (synthetic) προσομοίωση VIO, διακριτή από τα modules που υποθέτουν πραγματικά δεδομένα EuRoC. Ο διαχωρισμός αυτός είναι σημαντικός: επιτρέπει να δοκιμαστεί ολόκληρη η αλυσίδα “αβεβαιότητα → συμπεριφορά” χωρίς να απαιτείται η λήψη μεγάλων αρχείων δεδομένων (τα EuRoC σύνολα δεδομένων μπορούν να ξεπερνούν αρκετά GB), κάτι κρίσιμο για αναπαραγωγιμότητα και για ταχύτητα ανάπτυξης/δοκιμών.

4.2 ros2_ws/src/aegis_vio/

Πρόκειται για έναν τυπικό χώρο εργασίας (workspace) ROS2 της οικογένειας `ament_python`, που περιέχει το υποπακέτο `aegis_vio/` (τους τρεις κόμβους), τον κατάλογο `config/` με τέσσερα αρχεία YAML παραμετροποίησης, τον κατάλογο `launch/` με το αρχείο εκκίνησης όλων των κόμβων, και τον κατάλογο `msg/` με τις δύο προσαρμοσμένες δηλώσεις μηνυμάτων. Η ύπαρξη αυτού του καταλόγου, ξεχωριστού από το `src/`, αντικατοπτρίζει την τυπική σύμβαση οργάνωσης χώρων εργασίας ROS2 (`catkin/ament workspace`), όπου ένα `ros2_ws/src/` φιλοξενεί έναν ή περισσότερα πακέτα ROS2 που εν συνεχεία χτίζονται (`build`) με το εργαλείο `colcon`.

4.3 scripts/

Ο κατάλογος αυτός περιέχει 13 αυτόνομα εκτελέσιμα σενάρια Python, που μπορούν να ταξινομηθούν σε τρεις κατηγορίες: (α) σενάρια “επίσημης” διεπαφής γραμμής εντολών με χρήση `argparse` (`run_euroc.py`, `run_simulation.py`, `evaluate_results.py`, `download_euroc.py`), σχεδιασμένα να καλούνται όπως περιγράφεται στο README και στην τεκμηρίωση `docs/`, και (β) σενάρια πειραματικής/διερευνητικής φύσης χωρίς CLI ορίσματα, με σκληρά κωδικοποιημένες (hard-coded) διαδρομές αρχείων (π.χ. `compute_visual_uncertainty.py`, `compute_multimodal_uncertainty.py`, `train_failure_predictor.py`), που αντιπροσωπεύουν μια αλυσίδα διαδοχικών πειραμάτων ανάλυσης δεδομένων προς τα τελικά πειράματα 004 έως 009 της τεκμηρίωσης. Η ύπαρξη αμφότερων των στυλ είναι αναμενόμενη σε ένα ερευνητικό αποθετήριο σε

ενεργό στάδιο εξερεύνησης, αλλά αποτελεί και ένα σημείο τεχνικού χρέους (technical debt), όπως αναλύεται στην Ενότητα 17.

4.4 tests/

Ο κατάλογος tests/ περιέχει 7 αρχεία δοκιμών pytest, ένα ανά βασικό module του src/ (dataset_loader, ekf, evaluator, feature_tracker, navigation, synthetic_vio, uncertainty), με σύνολο 16 ρητά ονομασμένων συναρτήσεων ελέγχου (test_*) που το README αναφέρει συνολικά ως “35 unit tests” -πιθανώς μετρώντας επιπλέον επιμέρους ισχυρισμούς (assertions) ή νεότερες δοκιμές που δεν περιλαμβάνονται σε αυτή την έκδοση του κλωνοποιημένου αποθετηρίου. Η ύπαρξη δοκιμών για κάθε αλγοριθμικό module είναι ένδειξη σχετικά υπεύθυνης μηχανικής πρακτικής για ένα ερευνητικό αποθετήριο αυτής της κλίμακας.

4.5 docs/

Ο κατάλογος docs/ συγκεντρώνει την τεκμηρίωση εγκατάστασης (installation.md), χρήσης (usage.md), του συνθετικού demo (simulation.md), του γενικού πειραματικού πρωτοκόλλου στα αγγλικά (experiments.md) και τεσσάρων συγκεκριμένων αναφορών πειραμάτων στα ελληνικά (experiment_004 έως experiment_009), τα οποία τεκμηριώνουν συγκεκριμένα τρεξίματα πάνω στο σύνολο δεδομένων EuRoC MH_01_easy/MH_05_difficult με συγκεκριμένα αριθμητικά αποτελέσματα. Αυτό το μείγμα αγγλικής τεκμηρίωσης οδηγού χρήσης και ελληνικής τεκμηρίωσης ερευνητικών πειραμάτων υποδεικνύει ότι η συγγραφέας τεκμηριώνει το “πώς να τρέξει κανείς το σύστημα” διεθνώς (αγγλικά) αλλά τα ίδια τα ερευνητικά ευρήματα/πειράματα σε προσωπική, πιθανότατα ακαδημαϊκή γλώσσα εργασίας (ελληνικά).

4.6 datasets/, models/, plots/, results/

Πρόκειται για καταλόγους εργασίας (working directories) που είναι σκόπιμα κενοί ή ελάχιστα γεμάτοι στο ίδιο το αποθετήριο Git (τα δεδομένα EuRoC δεν αποθηκεύονται στο Git, όπως δηλώνεται ρητά στο docs/installation.md: “Datasets are not committed to GitHub”), αλλά γεμίζουν δυναμικά κατά την εκτέλεση των σεναρίων: datasets/ φιλοξενεί τα κατεβασμένα σύνολα δεδομένων EuRoC, models/ προορίζεται για αποθηκευμένα εκπαιδευμένα μοντέλα μηχανικής μάθησης, results/ συγκεντρώνει τα CSV αποτελέσματα κάθε πειράματος (π.χ. feature_quality/, visual_uncertainty/, multimodal_uncertainty/, navigation_modes/, failure_prediction/, simulation/), και plots/ τις αντίστοιχες οπτικοποιήσεις σε PNG. Αυτή η οργάνωση -ξεχωριστά δέντρα results/ και plots/ ανά πείραμα- διευκολύνει την αναπαραγωγικότητα και αποτρέπει την επικάλυψη αρχείων ανάμεσα σε διαφορετικά πειράματα.

4.7 web/

Ο κατάλογος `web/` περιέχει μια αυτόνομη, καθαρά client-side εφαρμογή (`simulation.html`, `simulation.js`, `styles.css`) χωρίς οποιαδήποτε εξάρτηση από διακομιστή (server) ή από τον κώδικα Python. Πρόκειται ουσιαστικά για ένα διαδραστικό, εκπαιδευτικό “δίδυμο” (conceptual twin) της λογικής `risk → mode → συμπεριφορά` που υλοποιείται στο `run_simulation.py`, αλλά εδώ ξαναγραφμένο εξ ολοκλήρου σε JavaScript με στόχο την άμεση, οπτική παρουσίαση μέσα σε φυλλομετρητή -προφανώς για παρουσιάσεις, ιστοσελίδα GitHub Pages (το αποθετήριο δηλώνει σχετικό σύνδεσμο panagiotagr.github.io/aegis-vio) ή εκπαιδευτική επίδειξη, χωρίς να απαιτεί από τον θεατή να εγκαταστήσει το περιβάλλον Python/ROS2.

4.8 .github/workflows/

Περιέχει ένα και μοναδικό αρχείο ροής εργασίας GitHub Actions, το οποίο όμως δεν εκτελεί δοκιμές μονάδας (CI testing) όπως θα περίμενε κανείς από το όνομα του καταλόγου, αλλά αποκλειστικά αναπτύσσει (deploy) το στατικό περιεχόμενο του αποθετηρίου (πιθανότατα τον φάκελο `web/`) σε GitHub Pages, ενεργοποιούμενο σε κάθε `push` στο κλάδο `main`. Η απουσία ενός πραγματικού CI pipeline που τρέχει αυτόματα το `pytest tests/` σε κάθε `pull request` αποτελεί ένα σαφές κενό ποιότητας, που σχολιάζεται περισσότερο στις Ενότητες 14 και 19.